

Framework Membedah Soal Algoritma LeetCode

Cara berpikir kritis sebelum memilih pattern, struktur data, traversal, pointer, atau teknik

Core idea

Jangan mulai dari pertanyaan: "ini pattern apa?". Mulai dari: bentuk datanya apa, target output apa, state yang perlu dibawa apa, geraknya ke mana, alat penyimpanannya apa, aturan skip/update/stop-nya apa, dan complexity yang diminta seberapa cepat.

Dokumen ini merapikan kategori berpikir algoritma supaya saat melihat soal LeetCode, lu tidak raba-raba.

Tujuannya bukan menghafal tanda-tanda soal, tapi membedah mesin masalahnya.

1. Setiap algoritma biasanya punya komponen apa?

Jawaban ringkas

Algorithm = input shape + goal + constraint + state + movement + memory/data structure + decision rule + stop condition + complexity.

No	Lapisan	Pertanyaan inti
1	Data shape	Bentuk datanya apa? Array, string, grid, tree, graph, linked list, interval, stream, list pilihan, atau sorted search space.
2	Goal / output	Yang diminta apa? Nilai, index, count, min/max, true/false, shortest path, semua kemungkinan, atau struktur baru.
3	Constraint	Ukuran input seberapa besar? Ini memberi target complexity: $O(n)$, $O(n \log n)$, $O(\log n)$, $O(2^n)$, dst.
4	State	Informasi minimum yang harus dibawa saat proses berjalan: posisi, window, path, visited, dp state, boundary, dsb.
5	Movement / traversal	Dari state sekarang pindah ke mana? $i++$, left/right, expand/shrink, neighbors, child, choice berikutnya, mid.
6	Data structure	Alat penyimpanannya apa? Set, Map, Queue, Stack, Heap, Trie, Union Find, DP table, parent map.
7	Decision rule / technique	Kapan skip, kapan add, kapan update answer, kapan shrink, kapan undo, kapan buang setengah search space.
8	Termination + edge case	Kapan berhenti? Goal ketemu, queue habis, path lengkap, $low == high$, input kosong, duplicate, blocked, target tidak ada.

2. Checklist cepat saat melihat soal

- Bentuk datanya apa? Garis lurus, grid, tree, graph, pilihan, atau ruang jawaban terurut?
- Output yang diminta apa? Nilai, count, min/max, path, length, true/false, atau semua solusi?
- Constraint-nya minta solusi secepat apa?
- State minimum yang harus gue simpan apa?
- Dari state sekarang, gerakanya ke mana?
- Butuh struktur data apa supaya gerakanya efisien?
- Aturan valid/invalid, skip, update, dan stop-nya apa?
- Edge case apa yang bisa bikin salah?

Kalimat kunci

Kalau lu belum bisa menjawab state + movement + decision rule, berarti lu belum benar-benar punya algoritma, baru punya feeling.

3. Kategori besar yang perlu lu lihat

Sudut pandang	Isi / contoh
Berdasarkan bentuk data	Linear: array/string/linked list. Hierarchical: tree. Network: graph. Spatial: grid/matrix. Choice space:

	decision tree/backtracking. Ordered search space: sorted array/answer range.
Berdasarkan cara bergerak	Sequential loop, two-sided left/right, window expand-shrink, layered BFS, deep DFS, undo backtracking, halving binary search.
Berdasarkan alat penyimpanan	Map, Set, Stack, Queue, Heap, Deque, Trie, Union Find, DP table, parent map.
Berdasarkan tujuan	Find, count, min/max, shortest path, all possibilities, validate, group/merge, optimize.
Berdasarkan teknik	Brute force, hashing, sorting, greedy, divide and conquer, dynamic programming, backtracking, graph traversal, binary search.
Berdasarkan complexity	$O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$, $O(n!)$.

4. Mapping cepat: bentuk data -> cara berpikir

Bentuk data	Cara berpikir natural
Array / List	Index, left/right, sliding window, prefix sum, sort, hash map.
String	Char, substring, frequency map, two pointers, trie, sliding window.
Grid / Matrix	Row/col, arah gerak, BFS/DFS, visited, shortest path, island/component.
Tree	Node, left/right child, recursion, DFS, BFS level order, depth.
Graph	Node-edge, adjacency list, visited, BFS/DFS, cycle, component, shortest path.
Linked List	Pointer next, dummy node, slow/fast, reverse, merge.
Intervals	Sort by start/end, overlap, merge, greedy.
List pilihan	Path, choice, constraint, goal, undo. Biasanya backtracking.
Sorted data / ordered answer	Boundary, mid, predicate $\text{can}(x)$. Biasanya binary search.

5. Data structure dipilih karena kebutuhan, bukan feeling

Kebutuhan	Alat yang cocok
Butuh ingat item pernah muncul?	Set
Butuh key -> value?	Map / HashMap
Butuh FIFO / level by level?	Queue
Butuh LIFO / undo / deep traversal?	Stack atau recursion
Butuh item min/max paling prioritas?	Heap / Priority Queue
Butuh range aktif yang bergerak?	left/right + counter/map
Butuh reconstruct path?	parent map
Butuh cek/gabung kelompok?	Union Find
Butuh prefix string?	Trie
Butuh simpan jawaban subproblem?	DP table / memo

6. Complexity sebagai kompas

Constraint	Arah solusi
$n \leq 10$	Brute force / backtracking mungkin aman
$n \leq 20$	Subset, bitmask, exhaustive search masih mungkin
$n \leq 100$	$O(n^3)$ kadang masih bisa
$n \leq 1.000$	$O(n^2)$ mungkin bisa

$n \leq 100.000$	Biasanya butuh $O(n \log n)$ atau $O(n)$
n sampai $1e9$ atau range besar	Butuh $O(\log n)$, math, atau binary search

7. Contoh bedah soal: Rotting Oranges

Soal

Grid berisi 0, 1, 2. Hitung menit sampai semua jeruk fresh menjadi busuk.

Lapisan	Isi
Data shape	Grid / matrix.
Goal	Waktu/layer sampai semua fresh orange kena.
Constraint	Butuh $O(m*n)$, setiap cell idealnya diproses sekali.
State	row, col, minute atau layer BFS.
Movement	4 arah: atas, bawah, kiri, kanan.
Data structure	Queue untuk multi-source BFS, plus count fresh.
Decision rule	Kalau neighbor fresh -> ubah jadi rotten, fresh--, masukkan queue.
Termination	Queue habis. Kalau fresh == 0 return menit; kalau masih ada fresh return -1.
Pattern	Multi-source BFS.

8. Contoh bedah soal: Word Ladder

Soal

Diberi beginWord, endWord, dan wordList. Cari panjang transformasi terpendek; tiap step cuma boleh ubah 1 huruf dan kata hasil harus valid.

Lapisan	Isi
Data shape	Graph tersembunyi dari kata-kata.
Goal	Shortest transformation length.
Constraint	WordList bisa besar, neighbor generation harus efisien.
State	current word + level.
Movement	Ubah 1 huruf menjadi word valid.
Data structure	Queue untuk BFS, Set dictionary, Set visited.
Decision rule	Kalau word baru ada di dictionary dan belum visited -> queue.
Termination	Kalau current == endWord return level; kalau queue habis return 0.
Pattern	BFS di implicit graph.

9. Contoh bedah: Menu makanan-minuman-dessert

Data asli

makanan = [nasi, mie]
 minuman = [teh, kopi]
 dessert = [puding, es krim]

Lapisan	Isi
Data shape	List of lists / kategori pilihan.
Goal	Semua paket menu valid.
Constraint	Total kemungkinan = $2 \times 2 \times 2 = 8$.
State	level kategori sekarang + path sementara.
Movement	Pilih satu item dari kategori sekarang, lanjut level + 1.
Data structure	path list dan result list.
Decision rule	path.add(choice) -> backtrack(level+1) -> path.removeLast().
Termination	Kalau level == jumlah kategori, simpan copy path.

Pattern	Backtracking / Cartesian product / eksplorasi ruang sampel.
----------------	---

```

final options = [
    ['nasi', 'mie'],
    ['teh', 'kopi'],
    ['puding', 'es krim'],
];

final path = <String>[];
final result = <List<String>>[];

void backtrack(int level) {
    if (level == options.length) {
        result.add(List<String>.from(path));
        return;
    }

    for (final choice in options[level]) {
        path.add(choice);      // choose
        backtrack(level + 1);  // explore
        path.removeLast();     // undo
    }
}

```

10. Contoh bedah: Binary Search on Answer

Soal

Cari minimum speed/capacity agar constraint terpenuhi. Search space-nya angka, bukan selalu array.

Lapisan	Isi
Data shape	Ruang jawaban angka yang terurut.
Goal	Minimum valid answer / boundary.
Constraint	Range besar, butuh $O(\log \text{range})$.
State	low, high, mid.
Movement	Tes mid, lalu buang setengah ruang pencarian.
Data structure	Boundary variable saja; bukan two pointers pattern.
Decision rule	can(mid)? true -> coba lebih kecil; false -> harus lebih besar.
Termination	low == high.
Pattern	Binary search on answer.

Catatan penting

Binary search memakai left/right sebagai boundary, bukan sebagai dua kandidat aktif seperti two pointers. Yang diuji adalah mid; hasil mid menentukan boundary mana yang digeser.

11. Template berpikir saat coding

```

// 1. Apa bentuk datanya?
// 2. Apa goal/output-nya?
// 3. Complexity target dari constraint?
// 4. State apa yang harus dibawa?
// 5. Movement/neighbor/next state-nya apa?
// 6. Data structure apa yang dibutuhkan?
// 7. Kapan skip/update/stop?
// 8. Edge case apa yang harus di-handle?

```

12. Pattern muncul setelah modelnya jelas

Jangan mulai dari pattern

Pertanyaan "ini pattern apa?" sering bikin buntu. Mulai dari bentuk ruang masalahnya dulu. Setelah state, movement, dan decision rule jelas, pattern biasanya muncul sendiri.

Bentuk masalah	Pattern yang natural
Butuh jawaban paling dekat/pendek dan semua langkah cost sama	BFS
Butuh menyelami semua yang nyambung / komponen / recursion natural	DFS
Butuh membangun semua kemungkinan dengan choose-explore-undo	Backtracking
Butuh menjaga range aktif yang bergerak	Sliding Window
Data sorted dan butuh pasangan/dua sisi	Two Pointers
Ruang jawaban terurut dan ada can(x)	Binary Search
Butuh ingatan cepat key/value atau pernah muncul	HashMap / HashSet
Butuh selalu ambil prioritas min/max	Heap
Butuh gabung/cek kelompok	Union Find
Butuh simpan hasil subproblem	DP

13. Lembar analisis soal LeetCode

Field	Isi saat latihan
Nama soal	
Data shape	
Goal/output	
Constraint dan target complexity	
State yang dibawa	
Movement / traversal	
Data structure	
Decision rule: skip/update/stop	
Edge cases	
Pattern akhir	

14. Ringkasan akhir

- Setiap algoritma bukan cuma "kode solusi". Ia punya bentuk data, state, gerak, alat penyimpanan, aturan keputusan, kondisi berhenti, dan target complexity.
- Traversal bukan cuma tree/graph; traversal bisa berarti cara menjelajahi ruang kemungkinan.
- Backtracking tree bukan data asli. Tree adalah visualisasi path kemungkinan dari data asli.
- Binary search bukan two pointers pattern. left/right hanyalah boundary untuk menentukan mid.
- Kalau ingin kritis saat lihat soal, selalu tanya: bentuk datanya apa, geraknya gimana, state yang gue bawa apa, dan kapan gue skip/update/stop?